Miguel Carlos
Angelo Calpoporo
Gavrie Clemente
Luis Biacora

CTF

Question 1

Previous Name and Current Name of the PC format: p1ka3frag{PR3V10USN4M3_CURR3NTN4M3}
Answer: p1ka3frag{mnmsrvc_P1K4-CH4N-Ds}

Approach

After mounting the E01 image using FTK Imager, we loaded the SYSTEM registry hive into Registry Explorer.

The following registry path was analyzed:

SYSTEM\ControlSet001\Control\ComputerName\ComputerName

This registry key stores the system's hostname configuration. Two relevant values were identified:

- **(Default)** → Previous computer name: `mnmsrvc`
- **ComputerName** → Current computer name: `P1K4-CH4N-DS`

These values were combined according to the challenge format to produce the flag.

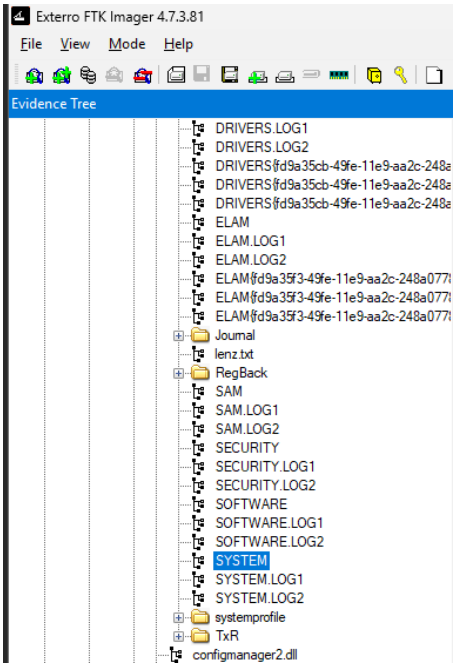


Figure 1. Exporting SYSTEM to open in Registry Explorer

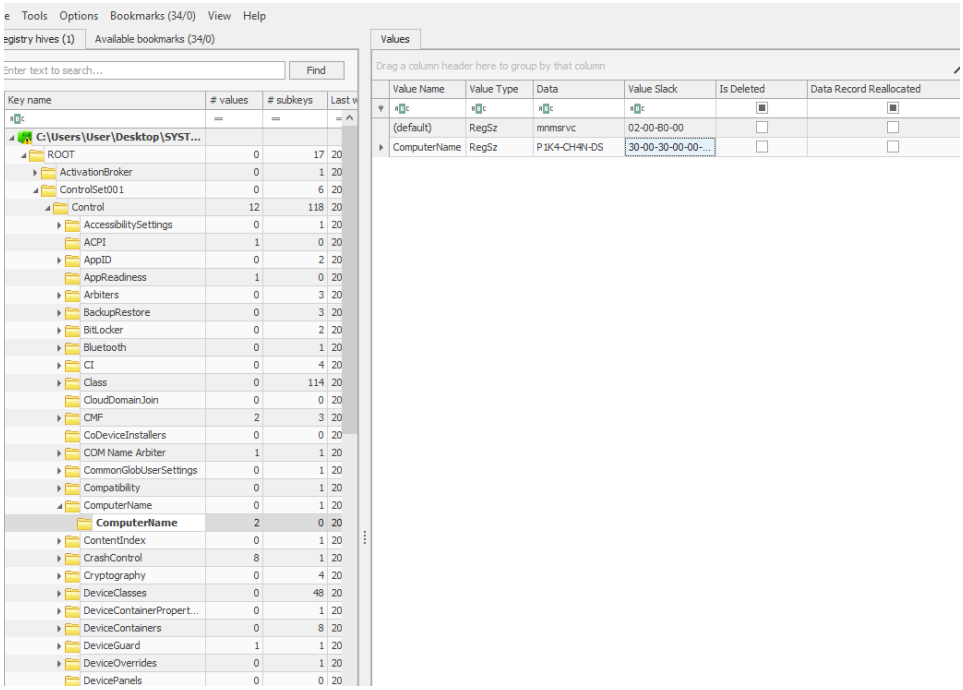


Figure 2. SYSTEM registry hive loaded in Registry Explorer showing the computer name entries.

Question 2

Operating System Version format: p1ka3frag{<OS><space><letters/number/float>-<Home/Pro/Enterprise>-<OSBuild>}
Answer: p1ka3frag{Windows 10 1903-Pro for Workstations-18362}

Approach

The **SOFTWARE registry hive** was examined using **Registry Explorer** to determine the operating system version.

The following registry path was inspected:

SOFTWARE\Microsoft\Windows NT\CurrentVersion

This key contains metadata describing the Windows installation. The following values were identified:

Registry Field	Value
ProductName	Windows 10 Pro for Workstations

ReleaseId	1903
CurrentBuild	18362
EditionID	ProfessionalWorkstation

From these values:

- OS Name → Windows 10
- Version → 1903
- Edition → Pro for Workstations
- Build → 18362

These values were formatted according to the required flag format.

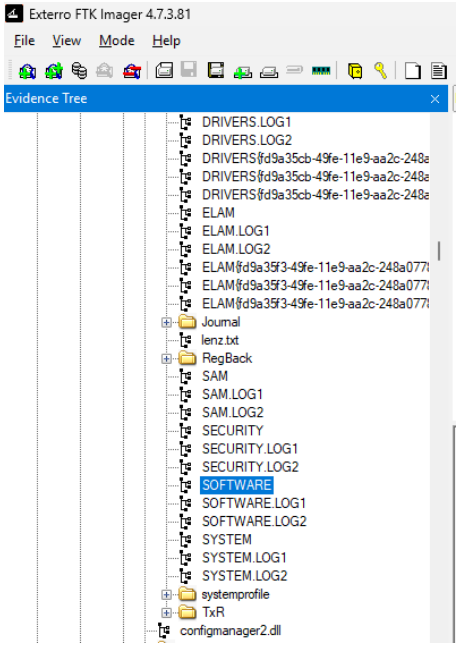


Figure 3. Exporting Software and opening in Registry Explorer

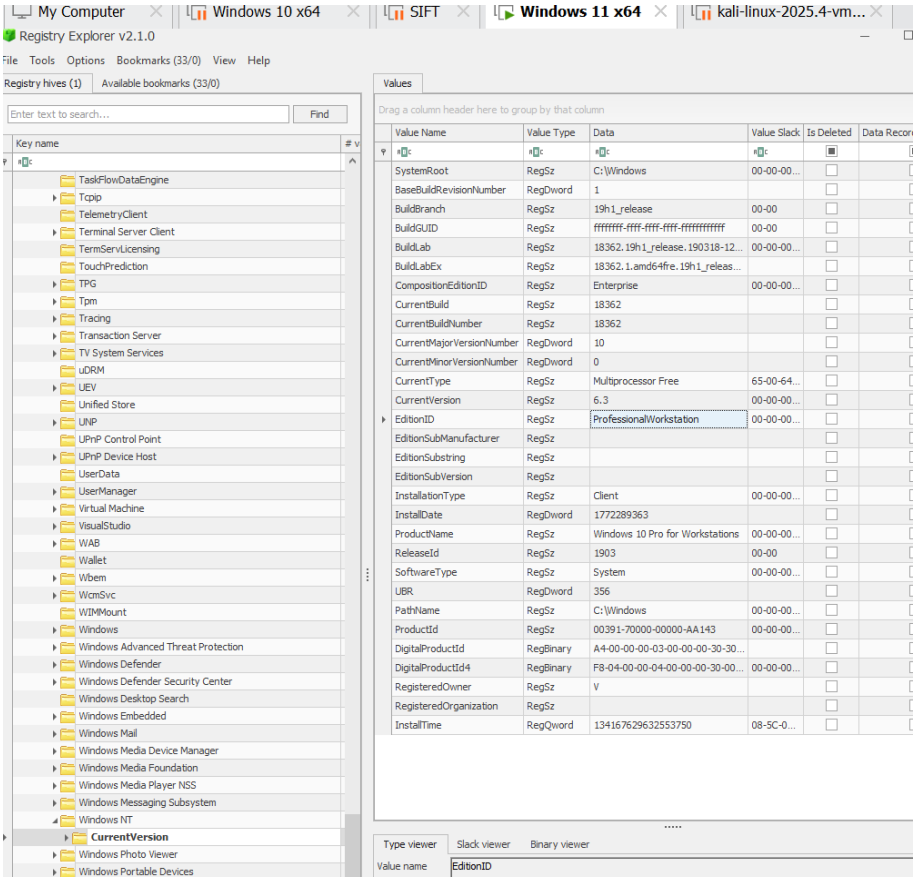


Figure 4. Registry Explorer showing the Windows operating system version details.

Question 3

What vulnerability is this machine possibly exploitable? Format: p1ka3frag{<OS><space><letters/number/float>-<VersionSpecificFromHAL>-<CVE-20xx-nnnn>}
Answer: p1ka3frag{Windows 10 1903-19h1-CVE-2020-0796}

Approach

The system version identified in the **SOFTWARE registry hive** indicated that the machine was running:

- **Windows 10 version 1903**
- **Build 18362**
- **Build branch: 19h1_release**

The build branch corresponds to the **Windows 10 1903 kernel/HAL version**.

To determine possible vulnerabilities affecting this version, the OS information was cross-referenced with:

- **NIST National Vulnerability Database (NVD)**
- **Microsoft Security Advisories**

The analysis revealed that Windows 10 version **1903** is affected by **CVE-2020-0796**, also known as **SMBGhost**, a remote code execution vulnerability in the SMBv3 protocol.

Question 4

What is the timezone of the machine with respect to UTC? Format: p1ka3frag{UTC<+/->xx}
Answer: p1ka3frag{UTC-08}

Approach

The **SYSTEM registry hive** was analyzed to determine the system's timezone configuration.

The following registry path was examined:

ControlSet001\Control\TimeZoneInformation

The value **TimeZoneKeyName** indicated **Pacific Standard Time**, and the **Bias** value of **480 minutes** represents the offset from Coordinated Universal Time (UTC). Converting 480 minutes results in an **8-hour difference**, meaning the system operates at **UTC-08**.

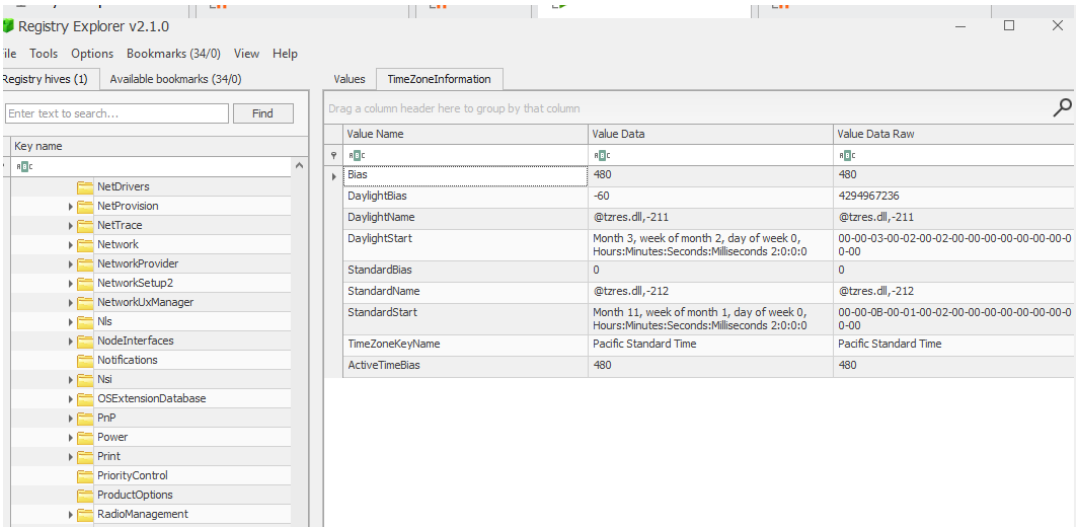


Figure 5. Timezone information obtained from the registry

Question 5

What vulnerability is this machine possibly exploitable? Format: p1ka3frag{<OS><space><letters/number/float>-<VersionSpecificFromHAL>-<CVE-20xx-nnnn>}
Answer:p1ka3frag{6}

Approach

The **SOFTWARE registry hive** was examined using **Registry Explorer**.

The following registry key was inspected:

SOFTWARE\Microsoft\Windows NT\CurrentVersion

The value **CurrentVersion** was identified as: **6.3**

Since the challenge requires only the integer value, the final answer corresponds to: **6**

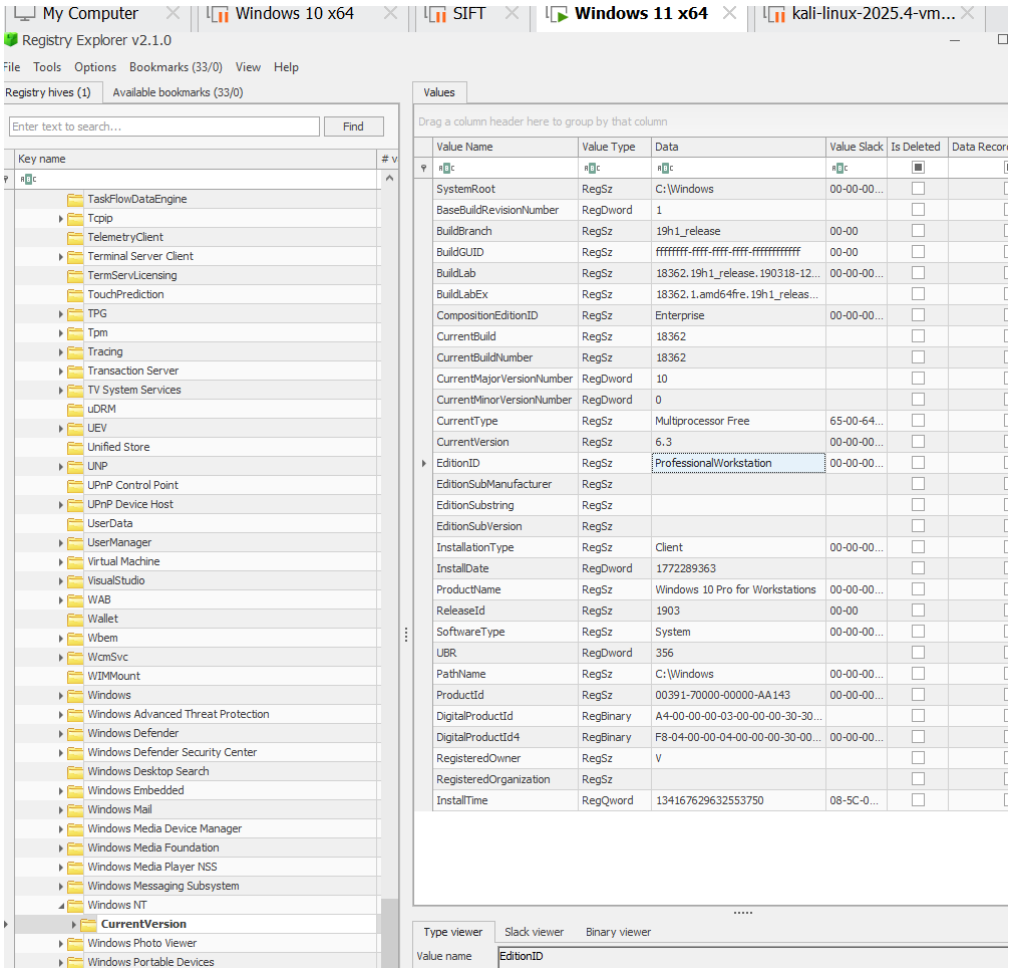


Figure 6. Registry Explorer showing the CurrentVersion field.

Question 6

A healing treat it may be. A healing threat it may see. A difficult thing to see. A random sight it may be. A loser we shall be. A winner we will see. A cursor you can see. A hidden key it may be. A single flag it may be. A tons of flags it will be.
Answer: p1ka3frog{DoeADeerAfemaleDeer05}

Approach

The forensic E01 image was examined using **FTK Imager**. Based on the challenge hint referencing the cursor position of a Word document, the **NTUSER.DAT** hive was analyzed using **Registry Explorer**. The registry path **Software\Microsoft\Office\16.0\Word\Reading Locations** was inspected, which stores metadata about previously opened Microsoft Word documents, including their file paths.

From this registry key, the entries **Document 0, Document 1, and Document 2** indicated that the user had previously opened the files **a.docx** and **b.docx**, which were located in the user's **Documents** directory.

Using this information, the forensic image was further examined in **FTK Imager**, and the following directory path was navigated:

```
Users
└── V
  └── Documents
```

Within this directory, the file **a.docx** were identified and extracted for analysis. Upon examining the contents of these documents, an **encoded CTF flag** was discovered within the document data.

The clue references a cursor and a hidden key, indicating a registry artifact related to Microsoft Word. Word stores the last cursor position of a document in the registry under the Word Reading Locations key within NTUSER.DAT. By examining this artifact, the cursor location of the document can be identified. The document contained many decoy flags, but the flag located at the recorded cursor position corresponds to the correct one. The cursor location pointed to the Appendix section of the document where the flag **p1ka3frog{DoeADeerAfemaleDeer05}** appears.

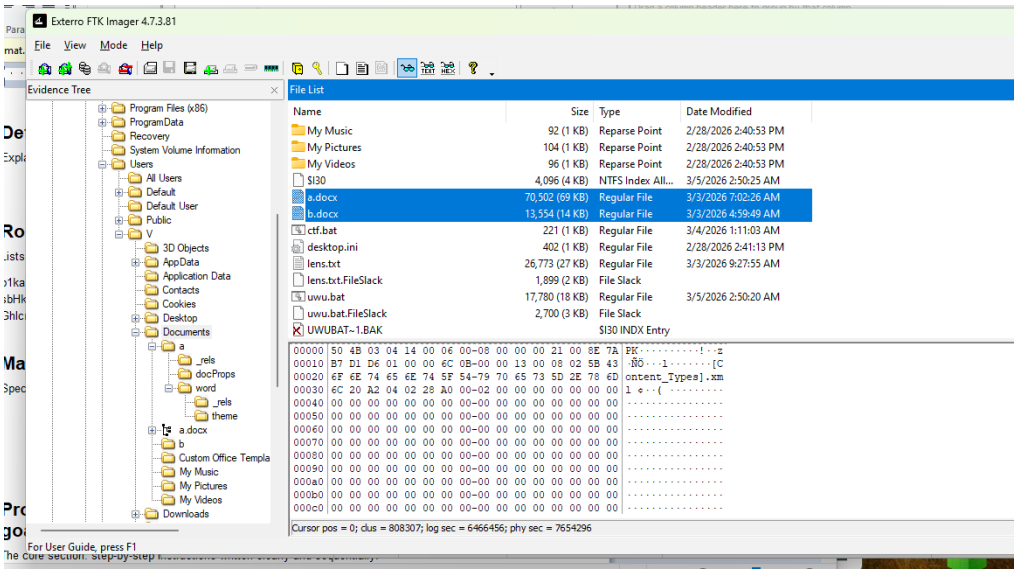


Figure 7. FTK Imager showing the recovered Word documents

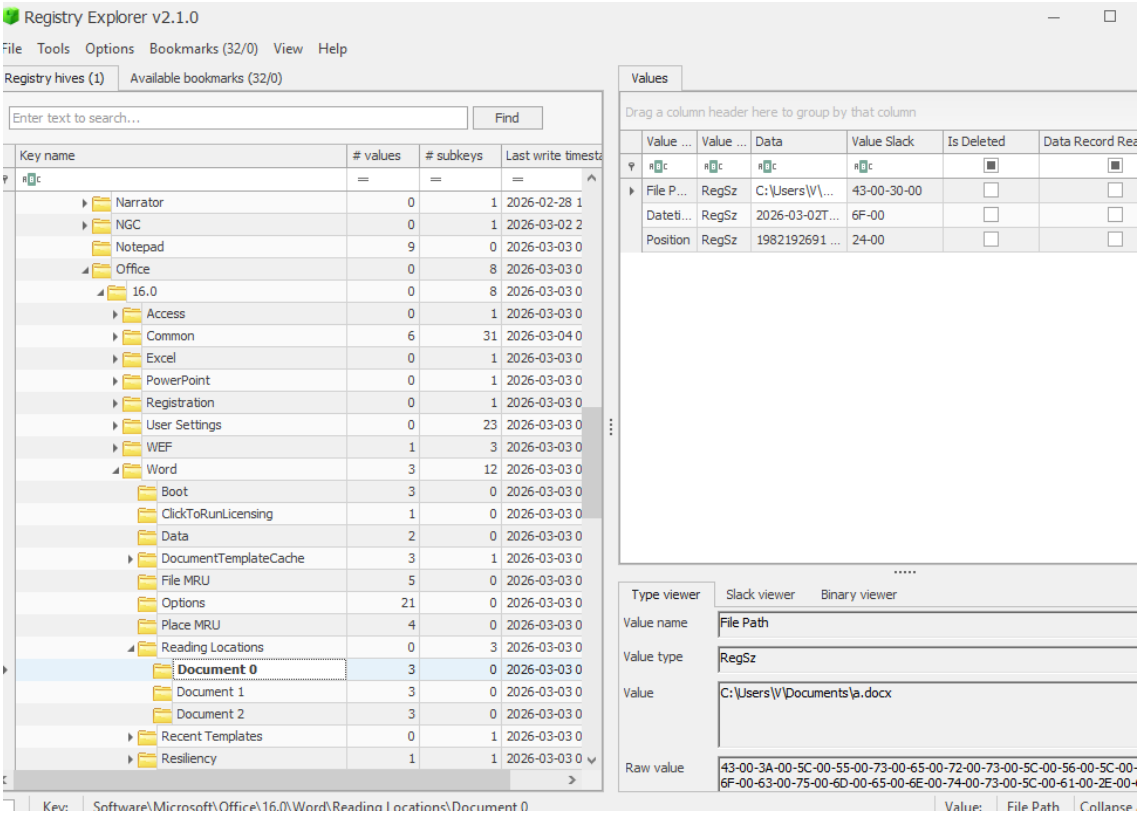


Figure 8. Registry Explorer showing the Word Reading Locations key

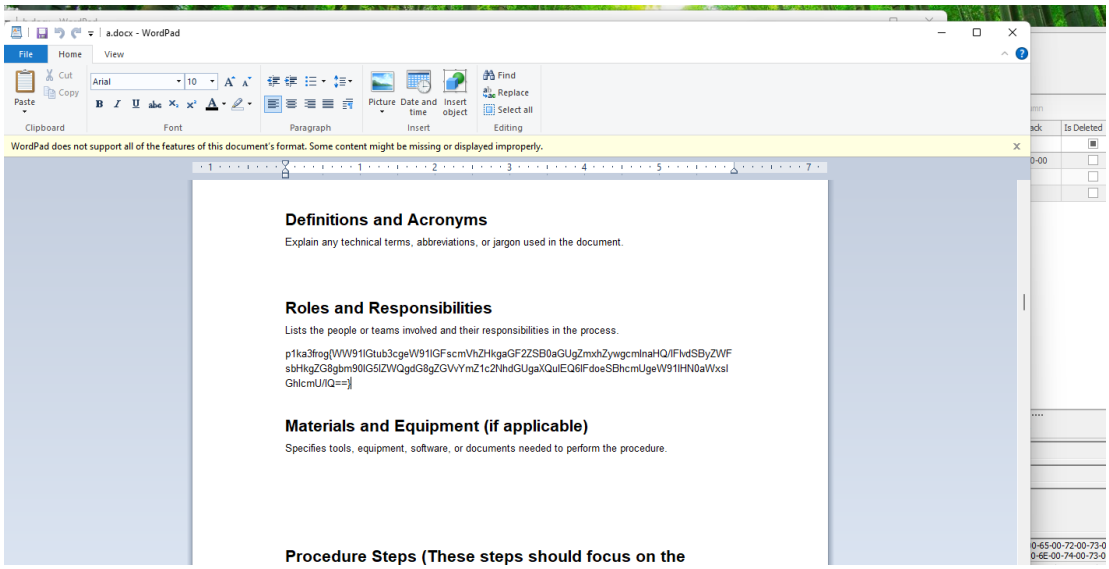


Figure 9. Extracted Word document containing the Base64-encoded CTF flag identified during analysis

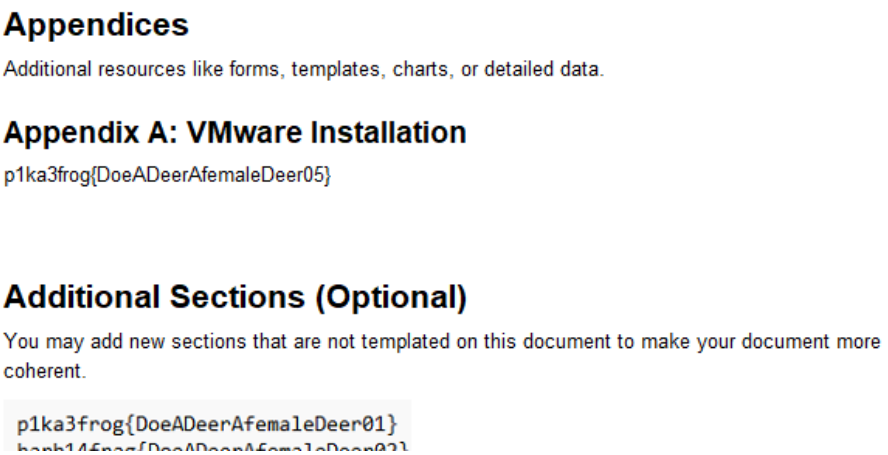


Figure 9.1 flag p1ka3frog{DoeADeerAfemaleDeer05} at Appendix A

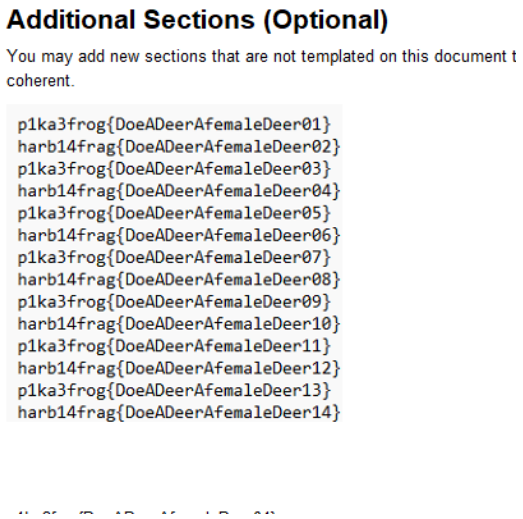
Question 7

You don't have to be a hero to see it. You just need patience in searching and not vibing. :)
Answer: p1ka3frag{DoeADeerAfemaleDeer01}

Approach

The clue suggests that the flag can be found through simple searching rather than complex decoding techniques. A keyword search for flag patterns such as `p1ka3frag` and `p1ka3frog` was performed across the artifacts. One entry contained a Base64-encoded message indicating that the correct flag was already present in the document and did not require deobfuscation. By patiently searching through the

visible entries, the first valid flag encountered was identified as p1ka3frag{DoeADeerAfemaleDeer01}, which is the correct flag.



Additional Sections (Optional)

You may add new sections that are not templated on this document t
coherent.

```
p1ka3frog{DoeADeerAfemaleDeer01}
harb14frag{DoeADeerAfemaleDeer02}
p1ka3frog{DoeADeerAfemaleDeer03}
harb14frag{DoeADeerAfemaleDeer04}
p1ka3frog{DoeADeerAfemaleDeer05}
harb14frag{DoeADeerAfemaleDeer06}
p1ka3frog{DoeADeerAfemaleDeer07}
harb14frag{DoeADeerAfemaleDeer08}
p1ka3frog{DoeADeerAfemaleDeer09}
harb14frag{DoeADeerAfemaleDeer10}
p1ka3frog{DoeADeerAfemaleDeer11}
harb14frag{DoeADeerAfemaleDeer12}
p1ka3frog{DoeADeerAfemaleDeer13}
harb14frag{DoeADeerAfemaleDeer14}
```

Figure 9.2 Word document section displaying multiple potential CTF flag strings discovered during analysis

Question 8

You All R Amazing people. But sometimes my LENs are dirty and some thorough cleaning you know like 6 (to) 7 times or 12. □ (use YARA)
Answer: lenz.txt

Approach

A YARA scan was performed on the mounted image to locate suspicious artifacts related to the clue. The wording “You All R Amazing” strongly suggests using YARA, while “LENs are dirty” points to the lenz.txt file. The scan repeatedly matched lenz.txt and its duplicate copies across multiple directories, all sharing the same MD5 hash, confirming they refer to the same planted artifact. Therefore, the clue identifies lenz.txt as the target file.

```
✅ Report saved: reports\scan_20260313_234519.json

C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py main.py G:\Users --rules rules\CTF.yar
============================================================
        MP1 MALWARE SCANNER - NSSECU3
============================================================
Target:      G:\Users
Rules:       rules\CTF.yar
Quarantine:  ./quarantine
Threads:     4
------------------------------------------------------------
2026-03-13 23:54:03,704 - src.yara_handler - INFO - Successfully compiled rules from rules\CTF.yar

🔍 Scanning started...
[CLEAN] NTUSER.DAT{fd9a35db-49fe-11e9-aa2c-248a07783950}.TM.blf | Binary data          | MD5: 40519eaf...
[CLEAN] NTUSER.DAT.LOG1 | Binary data          | MD5: 3cad38df...
[MALICIOUS] lenz.txt        | Binary data          | MD5: 36cd844c...
[CLEAN] NTUSER.DAT.LOG2 | ASCII text           | MD5: d41d8cd9...
[CLEAN] desktop.ini     | Binary data          | MD5: 6b1a6a99...
[MALICIOUS] lenz.txt        | Binary data          | MD5: 36cd844c...
[MALICIOUS] P.txt           | Binary data          | MD5: 36cd844c...
[CLEAN] desktop.ini     | Binary data          | MD5: 7220fad5...
[MALICIOUS] lenz.txt        | Binary data          | MD5: 36cd844c...
[CLEAN] NTUSER.DAT      | Binary data          | MD5: e2950b94...
[CLEAN] desktop.ini     | Binary data          | MD5: 2971c89b...
[MALICIOUS] V.txt           | Binary data          | MD5: 36cd844c...
[MALICIOUS] lenz.txt        | Binary data          | MD5: 36cd844c...
[CLEAN] desktop.ini     | Binary data          | MD5: dc723b85...
[CLEAN] desktop.ini     | Binary data          | MD5: ec659b64...
[MALICIOUS] lenz.txt        | Binary data          | MD5: 36cd844c...
```

Figure 10. Scanning of Files

```
[CLEAN] 03 - Computer Management.lnk | Binary data          | MD5: 9a97b14f...
[CLEAN] 04-1 - NetworkStatus.lnk | Binary data          | MD5: 353c6c0e...
[CLEAN] 05 - Device Manager.lnk | Binary data          | MD5: 3f72e097...
[CLEAN] 06 - SystemAbout.lnk | Binary data          | MD5: 6c46c4bf...
[CLEAN] 07 - Event Viewer.lnk | Binary data          | MD5: 37cef1a4...
[CLEAN] 08 - PowerAndSleep.lnk | Binary data          | MD5: c0be3fac...
[CLEAN] desktop.ini     | ASCII text           | MD5: 2b0c0eec...
[MALICIOUS] lenz.txt        | Binary data          | MD5: 36cd844c...
[MALICIOUS] lenz.txt        | Binary data          | MD5: 36cd844c...
[CLEAN] 09 - Mobility Center.lnk | Binary data          | MD5: 0119951f...
[CLEAN] 10 - AppsAndFeatures.lnk | Binary data          | MD5: 22e365a1...
[MALICIOUS] DefaultLayouts.xml | XML document         | MD5: 680c9fa8...

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     129
Malicious found:   61
Quarantined:       0
Errors:            0
Scan time:         2026-03-13 23:54:23.201732 seconds

✅ Report saved: reports\scan_20260313_235402.json
```

Figure 11. Scan Results

CTF.yar U × | scan_20260313_235402.json U | reporter.py MP2_MALWARE_SCANNER\... | quarantine.py MP2_MALWARE_SC

MP1_MALWARE_SCANNER > rules > CTF.yar

```
rule ctf_markers
{
    strings:
        $a = "p1ka3" ascii wide nocase
        $b = "frag{" ascii wide nocase
        $c = "frog{" ascii wide nocase
        $d = "Make sure to find the right one" ascii wide nocase
        $e = "It is hidden somewhere in the whole disk" ascii wide nocase
        $f = /LENZ[CDE]{20,}/ ascii wide nocase

    condition:
        any of them
}

rule find_real_p1ka_flag
{
    strings:
        $frag = /p1ka3frag\{[^}]+\}/ ascii wide nocase
        $frog = /p1ka3frog\{[^}]+\}/ ascii wide nocase

    condition:
        any of them
}
```

Figure 12. YARA rule used